

Report for assignment 2

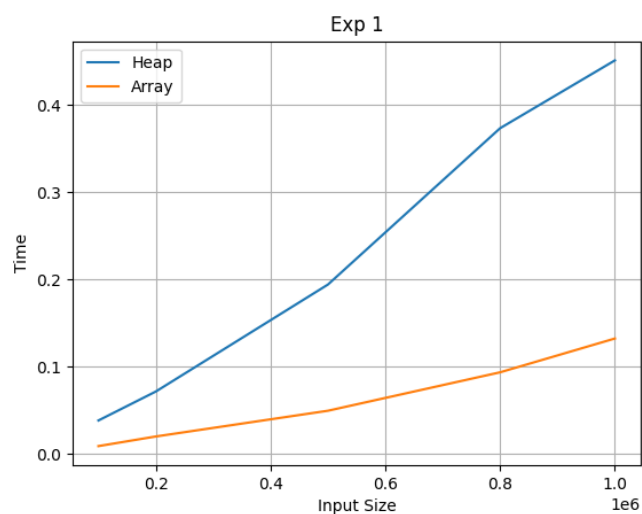
Student name: Afnan Alkhaibari
Student ID: 1718597

- **Experiment Environment:**

1. **CPU frequency:** 13th Gen Intel (R) Core (TM) i7-13620H
2. **The size of memory:** 16.0 GB
3. **Operating System:** Windows 11
4. **The programming language:** python
5. **IDE:** PyCharm
6. **Libraries used:** random, time, matplotlib

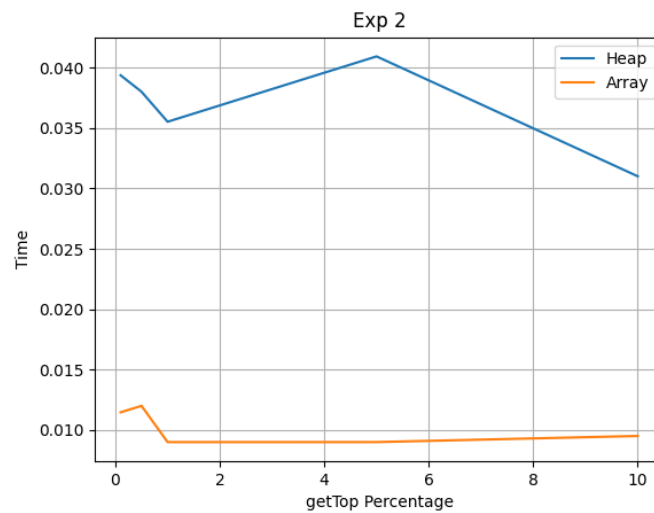
- **Analysis of the Experimental Results Experiment:**

1. **Exp 1: Time vs Length Push-only seque:**



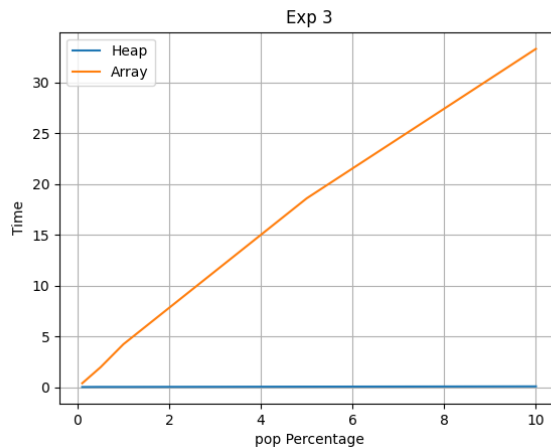
we studied the running time of push operations only, as the input size increased from 0.1M to 1M. From the first diagram, we notice that the time increases gradually for both structures as the size increases, but the array clearly outperformed the heap and was faster across all sizes. The reason for this is that the push operation in the array takes a constant time of $O(1)$ because we only append the new element at the back of the array and update the maximum element's index, making the overall complexity for a sequence of length L linear and equal to $O(L)$. In contrast, for every push operation, the heap needs to reorder the elements by invoking the bubble-up function, which takes a logarithmic time of $O(\log n)$. Therefore, the overall complexity for the heap is $O(L \log L)$, which explains why the array was practically faster in this experiment.

2. Exp 2: Time v.s. getTop Percentage:



We fixed the input size at 1M and varied the in expectation percentage of `getTop()` operations from 0.1% to 10% to see its effect on time. The diagram shows that the overall running time is very low for both structures, and as the percentage increases, the time tends to stabilize or slightly decrease. This similar performance is because the `getTop()` operation is very fast and takes a constant time of $O(1)$ in both structures; the heap reads directly from the first position in the array, and the array competitor returns the value using the pre-stored `i_max` index without any searching. `getTop()` operations are added to the sequence, the number of push operations naturally goes down. Since it is just a quick read operation that doesn't change anything in the data structure, it takes almost no time. This is why the total running time decreases, and the curve flattens out as the percentage increases.

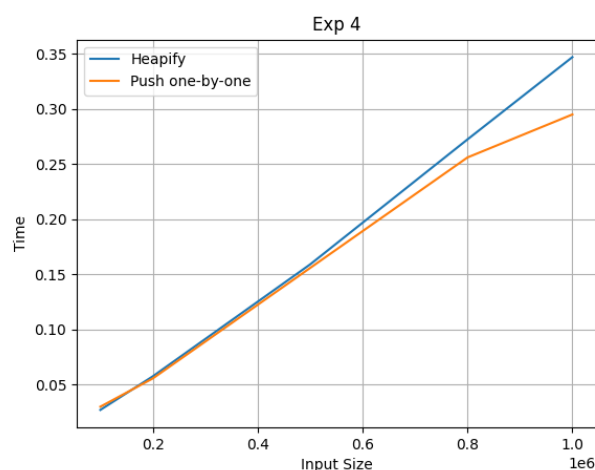
3. Exp 3: Time v.s. pop Percentage:



We tested what happens when we increase the percentage of pop() operations from 0.1% to 10% while keeping the total number of operations fixed. the graph shows us, the difference between the two structures. The array gets much slower as the percentage increases and reaches about 33 seconds at 10%, while the heap stays almost flat and every time the array, when we remove the maximum value, we need to go through all the elements to find the next maximum again. This is let the operation slow since it takes $O(n)$ each time.

now the heap doesn't need to scan everything. It only fixes its structure using bubble-down, which is faster and takes $O(\log n)$. Because of that, the heap handles pop operations much better when they are frequent.

4. Exp 4: Heapify v.s. Push-One-by-One:



We compared two methods for constructing the max-heap: the Heapify() function from bottom to top and the iterative insertion method. From the diagram, we notice a close similarity in time between the two methods, but upon reaching the

largest size of 1M, the iterative method Push one-by-one() outperformed the Heapify() method and became slightly faster, which is considered an interesting result because the theoretical worst-case analysis states that Heapify() is faster with a complexity of $O(n)$ compared to iterative insertion, which gives $O(n \log n)$. The data's randomness was the reason for this outcome. When elements were inserted one by one, most stayed at the bottom levels, rarely needing to move up. The expected number of bubble-up steps in the average case is a constant $O(1)$, so the actual running time was very close to $O(n)$. The data's randomness was the reason. This happens because the data is completely random, causing most elements to stay at the bottom levels and rarely move up. The expected number of bubble-up steps in the average case is a constant $O(1)$, so the actual running time was very close to $O(n)$.

- **Conclusion:**

Through the four experiments we conducted, we were able to understand how both structures work and identify the strengths and weaknesses of each in real-world scenarios. The results showed that a regular array is an excellent and very fast option in dynamic programs that rely on push operations and read only the largest element `getTop()` without deletion, because it maintains a constant and stable time throughout execution. However, once deletion operations `pop()` is introduced to the system, the array's efficiency collapses completely, and it becomes very slow because it must perform a full linear check with each deletion to find the new largest element. This is where the important job of Max-Heap() becomes apparent, as it overwhelmingly outperforms in this ever-changing context thanks to its ability to reduce deletion time to a simple logarithmic time. Finally, the experiments showed that actual performance in programming languages like Python may differ slightly from theoretical analysis of worst-case scenarios. We saw how the nature of random data and the cost of frequent self-calls affected the speed of heap construction methods, making single-input heaps practically superior to Heapify() by a small margin.